104. Указатели на функции

Перед вы урокам, что указатель – это переменная, которая содержит адрес другой переменной. Указатели на функции работают по аналогичному принципу. Но есть несколько нюансов, о которых мы сейчас и поговорим.

Оглавление:

1. Указатели на функции
2. Присваивание функции указателю на функцию
3. Вызов функции через указатель на функцию
4. Передача функций в качестве аргументов других функций
5. Указатель на функцию в аргументе
6. Использование typedef для упрощения синтаксиса
7. Использование auto begin() и end() в C++11
8. Заключение
9. Тест

Указатели на функции

Рассмотрим следующую простую функцию:

```
void foo()
{
    cout << "Foo!" << endl;
}
```

Адресом функции foo() – в нашем примере, Foo!() может быть получен с помощью оператора &, как и для переменной или массива. Например, следующий код выведет на экран адрес функции foo():

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    cout << &foo << endl;
    return 0;
}
```

Адрес функции foo() выглядит примерно следующим образом:

```
0x401000
```

Указатели на функции объявляются аналогично указателям на другие переменные. Например, следующая строка объявляет указатель на функцию foo. Если мы попытаемся вывести адрес функции, на который данный указатель на его адресе, мы получим тот же адрес, что и выше:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    cout << pFoo << endl;
    return 0;
}
```

Адрес функции foo() выглядит следующим образом:

```
0x401000
```

Как и в случае обычных переменных указатели на функции можно также инициализировать и присваивать друг другу. Например, следующая строка инициализирует указатель pFoo2 адресом, который хранится в указателе pFoo:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    return 0;
}
```

Адреса функции foo() выглядят следующим образом:

```
0x401000
0x401000
```

Также, как и в случае обычных переменных, функции можно присваивать друг другу. Например, следующая строка присваивает адрес функции foo() переменной pFoo:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    return 0;
}
```

Адреса функции foo() выглядят следующим образом:

```
0x401000
0x401000
```

Присваивание функции указателю на функцию

Функции можно также присваивать переменным, формирующим указатели на функции. Для этого вместо скобок после имени функции, мы пишем только имя функции:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    return 0;
}
```

Адреса функции foo() выглядят следующим образом:

```
0x401000
0x401000
```

Также, если переменная, которая является указателем на функцию, имеет тот же тип, что и функция, которую мы можем вызвать, то мы можем вызвать функцию, на которую ссылается переменная-указатель, вместо вызова функции foo():

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    return 0;
}
```

Адреса функции foo() выглядят следующим образом:

```
0x401000
0x401000
```

Вызов функции через указатель на функцию

Для того, чтобы вызвать функцию, на которую ссылается переменная-указатель, нужно использовать оператор вызова функции, а перед ним – имя переменной-указателя, заключенное в скобки:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    return 0;
}
```

Вывод на экран:

```
0x401000
0x401000
Foo!
```

Также, если переменная-указатель имеет аргументы, то мы можем передать аргументы, которые требуются функции, на которую ссылается переменная-указатель:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    (*pFoo2)(5, 7);
    return 0;
}
```

Вывод на экран:

```
0x401000
0x401000
Foo!
```

Передача функций в качестве аргументов других функций

Одно из самых полезных свойств указателей на функции заключается в передаче их аргумента – то есть передача функции другой функции в качестве аргумента. Для этого, в списке параметров функции, которая принимает другую функцию, мы должны определить переменную-указатель, которая будет принимать адрес функции, которая будет передана.

Например, следующая функция foo() принимает в качестве аргумента функцию, которая не принимает никаких аргументов и не возвращает никакого значения:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    (*pFoo2)(5, 7);
    return 0;
}
```

Для того, чтобы передать функцию foo() в качестве аргумента, мы можем использовать имя функции foo() в качестве аргумента:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    (*pFoo2)(5, 7);
    return 0;
}
```

Вывод на экран:

```
0x401000
0x401000
Foo!
```

Для того, чтобы вызвать функцию, которая является аргументом, нужно использовать оператор вызова функции, а перед ним – имя переменной-указателя, которая ссылается на функцию, которая передается в качестве аргумента:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    (*pFoo2)(5, 7);
    return 0;
}
```

Вывод на экран:

```
0x401000
0x401000
Foo!
```

Указатель на функцию в аргументе

Если мы хотим передать функцию в качестве аргумента, то нам нужно определить переменную-указатель, которая будет принимать адрес функции, которая будет передана. Например, следующая функция foo() принимает в качестве аргумента функцию, которая не принимает никаких аргументов и не возвращает никакого значения:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    (*pFoo2)(5, 7);
    return 0;
}
```

Вывод на экран:

```
0x401000
0x401000
Foo!
```

Для того, чтобы вызвать функцию, которая является аргументом, нужно использовать оператор вызова функции, а перед ним – имя переменной-указателя, которая ссылается на функцию, которая передается в качестве аргумента:

```
void foo()
{
    cout << "Foo!" << endl;
}

int main()
{
    int (*pFoo)();
    pFoo = &foo;
    int (*pFoo2)(int, int) = pFoo;
    cout << pFoo << endl;
    cout << pFoo2 << endl;
    (*pFoo)();
    (*pFoo2)(5, 7);
    return 0;
}
```

Вывод на экран:

```
0x401000
0x401000
Foo!
```

Заключение

Таким образом, указатели, которые ссылаются на адреса функций, ведут себя почти так же, как и указатели на другие переменные. Для вызова функции, на которую ссылается переменная-указатель, нужно использовать оператор вызова функции, а перед ним – имя переменной-указателя, заключенное в скобки. Также, переменные-указатели на функции могут быть переданы в качестве аргумента другой функции. Получив в качестве аргумента переменную-указатель на функцию, мы можем вызвать функцию, на которую ссылается переменная-указатель, с помощью оператора вызова функции.

Тест

Задание №1

Напишите функцию-сравнения, которая возвращает отрицательное значение, если первый параметр меньше второго, ноль, если параметры равны и положительное значение в противном случае.

Ответ 1.0

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;
}
```

Выбрасывать функцию cmp() целиком, только потому, что должна передаваться для использования переменная-указатель, неправильно, поэтому:

Ответ 1.1

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;
}

int (*pCmp)(int, int);
pCmp = &cmp;
int (*pCmp2)(int, int) = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

И создавать указатель сначала – неправильно, так как сначала нужно объявить переменную-указатель, а потом присваивать ей адрес функции, поэтому:

Ответ 1.2

```
int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

А объявлять в функции-сравнения функцию для работы неправильно:

Ответ 1.3

```
int cmp(int, int)
{
    int (*pCmp)(int, int);
    int (*pCmp2)(int, int);
    pCmp = &cmp;
    pCmp2 = pCmp;
    cout << pCmp << endl;
    cout << pCmp2 << endl;
    (*pCmp)(5, 7);
    (*pCmp2)(5, 7);
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;
}
```

Остается только вариант:

Ответ 1.4

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;
}

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Задание №2

Теперь добавьте в данную программу, которая вы написали в те задание, чтобы перевести метод на дополнительный аргумент.

Ответ 2.0

Все создаваемые переменные-указатели на функции, которые получают дополнительный аргумент:

Ответ 2.0а

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

И сравнение при чего вместе:

Ответ 2.0б

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Задание №3

Напишите функцию-сравнения, которая принимает в качестве аргумента функцию, которая принимает два аргумента и возвращает целое значение.

Ответ 3.0

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Все создаваемые переменные-указатели:

Ответ 3.0а

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Все сравнения при чего вместе:

Ответ 3.0б

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Задание №4

Напишите функцию **qsort()** Вывод:

Ответ 4.0

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Все создаваемые переменные-указатели:

Ответ 4.0а

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```

Все сравнения при чего вместе:

Ответ 4.0б

```
int cmp(int, int)
{
    if (x < y)
        return -1;
    if (x == y)
        return 0;
    return 1;

int (*pCmp)(int, int);
int (*pCmp2)(int, int);
pCmp = &cmp;
pCmp2 = pCmp;
cout << pCmp << endl;
cout << pCmp2 << endl;
(*pCmp)(5, 7);
(*pCmp2)(5, 7);
```